Guía para crear tus propias funciones de listas

1. Pasaje de parámetros

- Por un lado vamos a encontrar **Nodo *lista**, a esto lo llamamos pasar por valor a la lista, Su uso lo debemos limitar a funciones que no requieren modificar la lista (mostrar, buscar, contar, etc).

- Por otro lado, tenemos **Nodo *&lista**, a esto lo llamamos pasar por referencia la lista, lo debemos usar siempre y cuando debamos modificar la lista de alguna forma (insertar, eliminar, modificar), ya que sin el **&**, los cambios al finalizar la función se pierden.

2. ¿Cómo recorro una lista?

Siempre que vayamos a recorrer una lista, debemos hacerlo de la siguiente forma:

```
Nodo *aux = lista;
While(aux != NULL){
	aux = aux->sgte; //para avanzar al siguiente nodo
}
```

Usamos siempre un aux para no recorrer la lista principal y perder los punteros.

3. Pasos para agregar un nodo a una lista.

Siempre que vayamos a modificar una lista, debemos hacer lo siguiente:

1) Crear el espacio de memoria para el nodo que vamos a agregar

   ```
   Nodo *nuevo_nodo = new Nodo();
   ```

2) Siguiente, debemos asignar el dato al nodo que creamos

   ```
   Nuevo_nodo->info = valor
   ```

3) Por último debemos acomodar el nodo en su lugar, recordemos que si debemos movernos en la lista, lo hacemos a través de una lista aux.

4. Pasos para quitar un nodo de la lista.

Si vamos a quitar los nodos. Debemos movernos hasta donde esté el nodo y borrar el aux con un **delete**. Acá va a depender muchísimo de la lógica del ejercicio más que de otra cosa. Por ejemplo, vamos a desarrollar una función para borrar el primer nodo de la lista

```
void eliminarPrimeroLista(Nodo *&lista){
	Nodo *aux = lista;
	int dato = aux->info;
	lista = aux->sgte;
	delete aux;
}
```

5. Casos borde (Corner cases)

Este es un tema muy importante ya que siempre puede perjudicarnos en algún punto. Debemos verificar siempre los siguientes casos

- ¿La lista está vacía? (**lista == NULL**)
- ¿La lista tiene un solo nodo? (**lista->sgte == NULL**)

Por otro lado si o si hay que asegurarnos que el puntero de la izquierda no sea NULL, ya que si hacemos un aux->sgte y aux es NULL el programa puede no compilar

6. Usar un ant y act.

Para insertar en el medio o eliminar un nodo específico

```
Nodo *actual = lista;
Nodo *ant = NULL;

while (actual != NULL && actual->info != valor_buscado) {
    ant = actual;// El anterior sigue al actual
    actual = actual->sgte; // El actual avanza
}
```

6. Estructura recomendada cuando debemos insertar un nodo en algún lugar dentro de nuestra lista (en medio de dos nodos por ejemplo).

1) Crear el nuevo_nodo y asignarle el info.
2) Buscar la posición
3) Re-conectar: nuevo_nodo->sgte = actual;
   - Si ant es NULL, el nuevo nodo es la nueva cabeza (lista = nuevo_nodo).
   - Sino, ant->sgte = nuevo_nodo.